Node.js — Day 24

Middleware Concepts

Day 24 Goal
Understand what middleware is, where it fits in a request flow, what **next()** means, how multiple middleware functions work together, and what happens when a request is not allowed to continue.

1. What is Middleware?

Middleware is a **function or layer that runs during the request flow**, usually before the final route handler. It can perform some work, decide whether the request should continue, or stop the request by sending a response.

Common jobs of middleware:

- Logging requests
- Checking authentication
- Validating incoming data
- Inspecting request information
- Measuring request time
- Rejecting a request when a condition is not satisfied

Real-world analogy
Think of an office security guard. A visitor arrives → the guard checks the ID → if everything is okay, the visitor is allowed to continue inside. The guard is like middleware; **next()** is the "continue" signal.

2. The Request Flow

Middleware sits in the path between the incoming request and the final handler.

```
Request
   ↓
Middleware 1
   ↓ next()
Middleware 2
   ↓ next()
Route Handler
   ↓
Response
```

A request can pass through **multiple middleware functions** before the final response is produced.

3. What does next() mean?

next() tells the request flow: *"This middleware is done. Continue to the next step."*

```
function logger(req, res, next) {
    console.log("Request received");
    next();
}
```

Memory trick
next() = Pass the request forward ➡■

4. Middleware Can Run in a Chain

More than one middleware function can participate in the same request flow. Each middleware can do its own job and then call **next()** to move forward.

```
function logger(req, res, next) {
    console.log("Logging request");
    next();
}

function checkAuth(req, res, next) {
    console.log("Checking authentication");
    next();
}
```

```
Request → logger → checkAuth → final handler → Response
```

The first middleware calls **next()**, so the second middleware gets a chance to run. The second middleware calls **next()**, so the final handler can send the response.

5. What if next() is Missing?

If middleware neither calls **next()** nor sends a response, the request can remain waiting. From the user's point of view, the request may appear to be stuck or hanging.

```
function logger(req, res, next) {
    console.log("Request received");
    // next() is missing
}
```

> **Important**
> A middleware function needs to either let the flow continue or otherwise finish/stop the request. Simply doing some work and then doing nothing can leave the request waiting.

6. Three General Things Middleware Can Do

Action	Meaning
Do something + **next()**	Perform middleware work and continue the request flow.
Send a response	Finish the request instead of continuing to another handler.
Pass an error	Signal that something went wrong; error handling is covered separately later.

7. Practical Node.js Example

The following example uses Node's built-in **http** module to manually demonstrate the middleware idea. Node's built-in HTTP module does not automatically provide Express's middleware system; here we are wiring the flow ourselves for learning.

```
const http = require("http");

function logger(req, res, next) {
    console.log("Request received");
    next();
}

const server = http.createServer((req, res) => {
    logger(req, res, () => {
        res.end("Hello from server");
    });
});

server.listen(3000);
```

Here, **logger()** does its job first. When it calls **next()**, the callback passed to it runs and sends the final response.

8. Two-Middleware Practical Example

```
function logger(req, res, next) {
    console.log("Logging request");
    next();
}

function checkAuth(req, res, next) {
    console.log("Checking authentication");
    next();
}

const http = require("http");

const server = http.createServer((req, res) => {
    logger(req, res, () => {
        checkAuth(req, res, () => {
            res.end("Welcome!");
        });
    });
});

server.listen(3000);
```

Execution order:

```
Request
  ↓
logger() → prints: Logging request
  ↓ next()
checkAuth() → prints: Checking authentication
  ↓ next()
res.end("Welcome!")
  ↓
Response: Welcome!
```

9. Mini Challenge — Solution

The challenge was to complete both middleware functions so that the request reaches the final response.

```
function logger(req, res, next) {
    console.log("Request received");
    next();
}

function checkAuth(req, res, next) {
    console.log("Checking authentication");
    next();
}
```

Why both next() calls matter
The first **next()** moves the request from **logger** to **checkAuth**. The second **next()** moves it from **checkAuth** to the final handler.

10. Final Assessment

Question	Correct answer
What is middleware?	**B** — A function/layer that runs during the request flow.
What does next() do?	**C** — Passes the request to the next step.
What can happen if middleware neither calls next() nor sends a response?	**A** — The request may hang.

Assessment result
3 / 3 correct ■ — Day 24 concepts and practical exercise completed.

11. Day 24 Cheat Sheet

Concept	Remember
Middleware	A function/layer that runs during the request flow.
next()	Tells the request flow to continue to the next step.
Middleware chain	Multiple middleware functions can run one after another.
Final handler	The step that ultimately sends the response in our examples.
Missing next()	If no response is sent either, the request can hang.
Node http example	Used here to manually demonstrate the middleware concept.

Mental model
Request → **Middleware** → **next()** → **Middleware** → **next()** → **Handler** → **Response**

Day 24 Complete ■
You completed the middleware concept, request flow, **next()**, middleware chaining, a practical Node.js example, a mini challenge, and the final 3/3 assessment.

Long-term learning note: Express will later provide a proper middleware system. Today's manual Node.js examples are meant to make the underlying request-flow idea clear first.